

DATABASE
management

Management Mobile Shop

BY JAFAR KHAN





INTRODUCTION

Database management in the Mobile Shop project is used to store and manage data such as mobiles, customers, employees, and orders in an organized way.

It uses tables with primary and foreign keys to maintain accuracy and make data retrieval easy.



OBJECTIVES

- To manage mobile brand information using the brands table.
- To store and organize mobile details using the mobiles table.
- To maintain customer information using the customers table.
- To manage employee details using the employees table.
- To record and track sales orders using the orders table.

Field	Type	Null	Key	Default
brand_id	int	NO	PRI	NULL
brand_name	varchar(50)	NO		NULL
country	varchar(30)	YES		NULL
founded_year	int	YES		NULL
support_email	varchar(50)	YES	UNI	NULL

BRAND

DATABASE SCHEMA

Field	Type	Null	Key	Default
customer_id	int	NO	PRI	NULL
full_name	varchar(50)	NO		NULL
phone	varchar(15)	YES	UNI	NULL
email	varchar(50)	YES	UNI	NULL
city	varchar(30)	YES		NULL
gender	varchar(10)	YES		NULL
registered_date	date	YES		NULL

CUSTOMER

Field	Type	Null	Key	Default
order_id	int	NO	PRI	NULL
customer_id	int	YES	MUL	NULL
mobile_id	int	YES	MUL	NULL
employee_id	int	YES	MUL	NULL
payment_mode	varchar(20)	YES		NULL
total_amount	int	YES		NULL
order_date	date	YES		NULL

ORDER

DATABASE SCHEMA

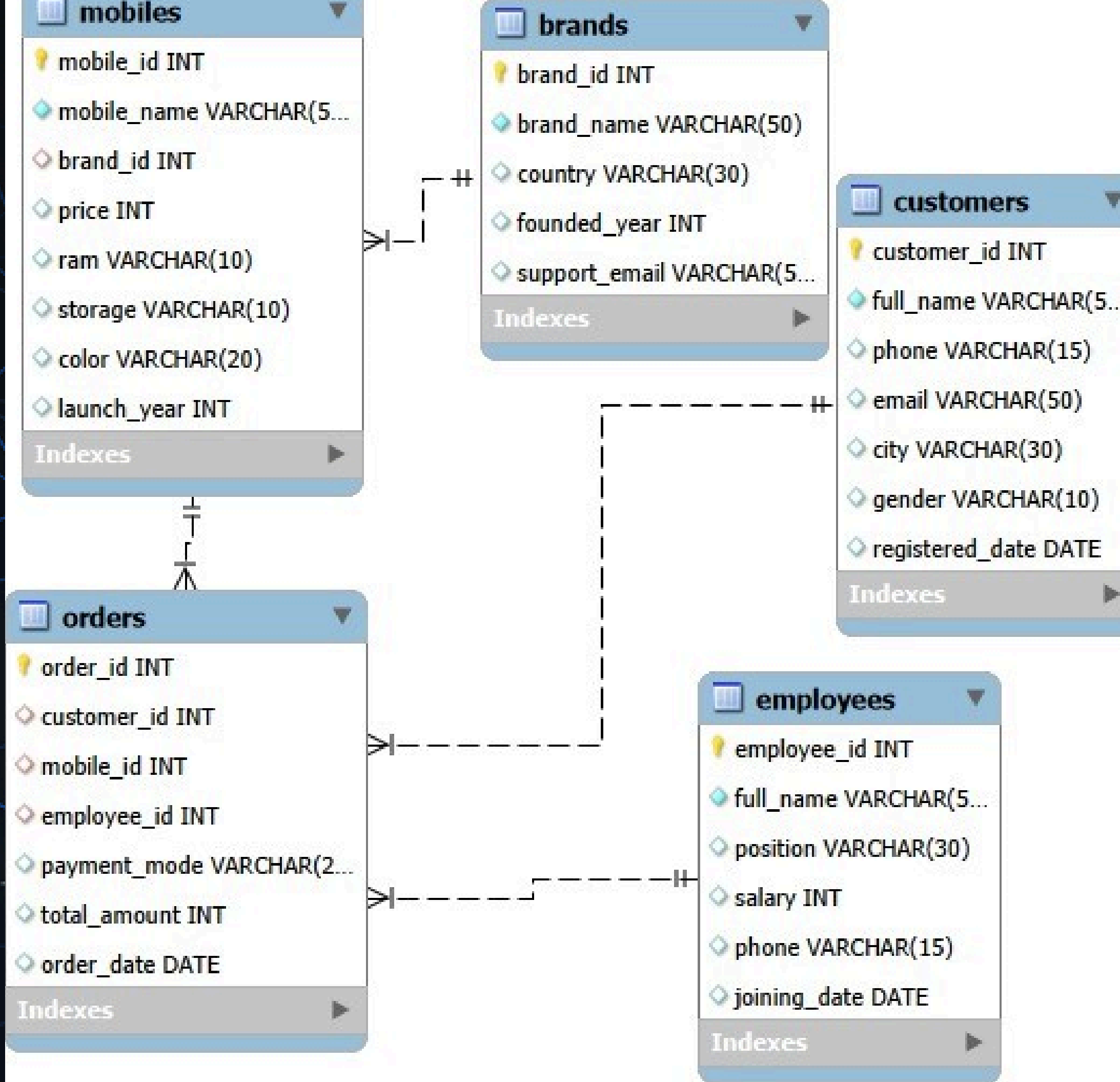
EMPLOYEE

Field	Type	Null	Key	De
employee_id	int	NO	PRI	NU
full_name	varchar(50)	NO		NU
position	varchar(30)	YES		NU
salary	int	YES		NU
phone	varchar(15)	YES	UNI	NU
joining_date	date	YES		NU

Field	Type	Null	Key	Default
mobile_id	int	NO	PRI	NULL
mobile_name	varchar(50)	NO		NULL
brand_id	int	YES	MUL	NULL
price	int	YES		NULL
ram	varchar(10)	YES		NULL
storage	varchar(10)	YES		NULL
color	varchar(20)	YES		NULL
launch_year	int	YES		NULL

MOBILE

ER DIAGRAM



QUERIES & OUTPUTS

```
124 -- INNER JOIN
125 -- display mobile name with brand name using INNER JOIN.
126 • SELECT m.mobile_name, b.brand_name
127 FROM mobiles m
128 INNER JOIN brands b ON m.brand_id = b.brand_id;
129
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	mobile_name	brand_name
	Galaxy S21	Samsung
	Galaxy A14	Samsung
	iPhone 13	Apple
	iPhone 11	Apple
	Redmi Note 12	Xiaomi
	Redmi 10	Redmi



```

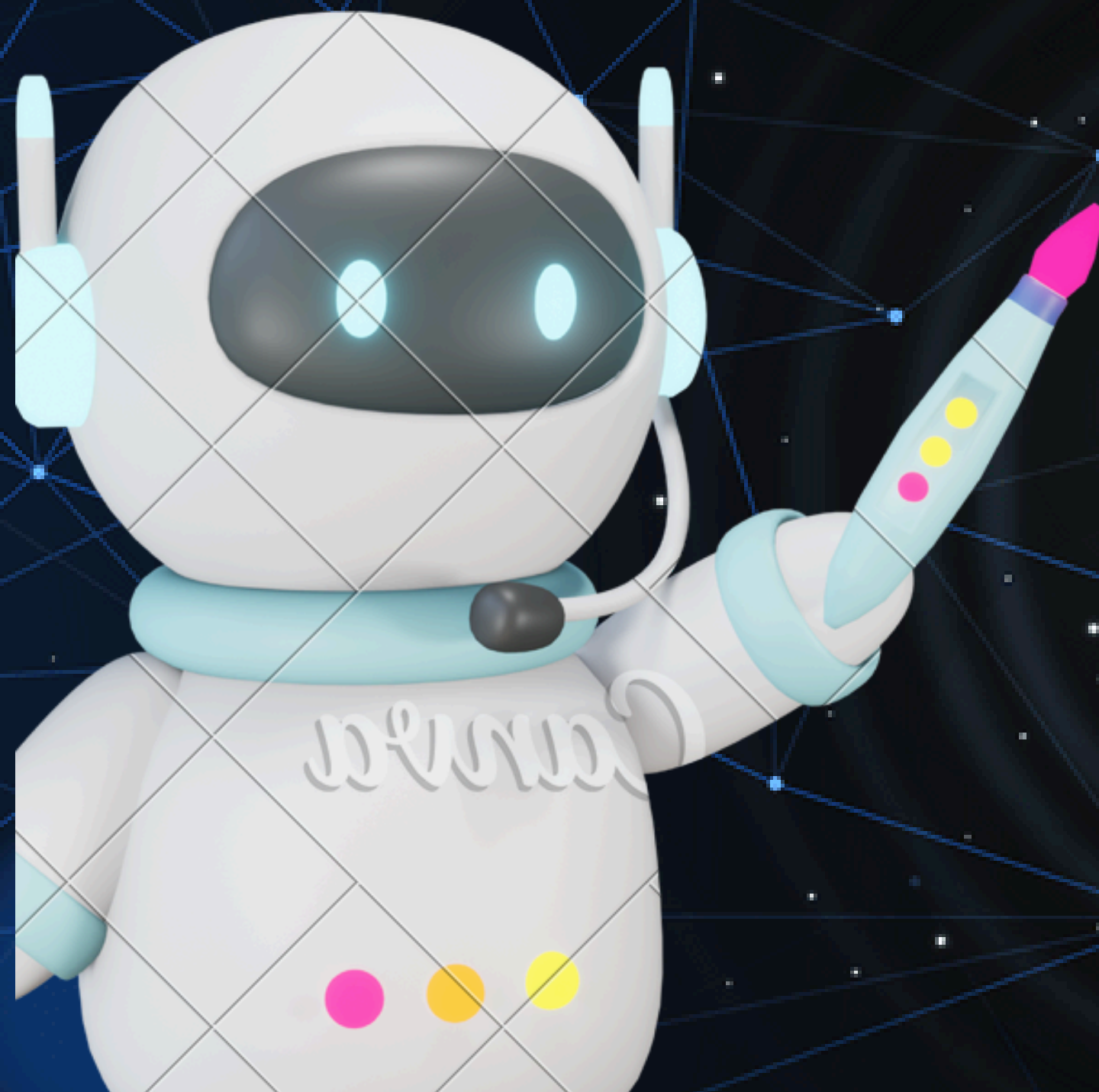
130  -- LEFT JOIN
131  -- display all customers and their order id using LEFT JOIN.
132  • SELECT c.full_name, o.order_id
133  FROM customers c
134  LEFT JOIN orders o ON c.customer_id = o.customer_id;
135

```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	full_name	order_id
+	Aman Verma	1
	Rahul Sharma	2
	Sneha Patil	3
	Pooja Singh	4
	Rohit Jain	5
	Neha Mehta	6
	Karan Malhotra	7
	Anjali Desai	8
	Vikas Yadav	9
	Priva Kannor	10





```
136      -- RIGHT JOIN
137      -- display all orders and customer names using RIGHT JOIN.
138  •    SELECT c.full_name, o.order_id
139      FROM customers c
140      RIGHT JOIN orders o ON c.customer_id = o.customer_id;
141
```

<

Result Grid ☐ Filter Rows: Export: ☐ Wrap Cell Content: ☐

	full_name	order_id
▶	Aman Verma	1
	Rahul Sharma	2
	Sneha Patil	3
	Pooja Singh	4
	Rohit Jain	5
	Neha Mehta	6
	Karan Malhotra	7
	Anjali Desai	8
	Vikas Yadav	9
	Priva Kannor	10

```

142      -- FULL OUTER JOIN
143      -- display all customers and all orders using FULL OUTER JOIN.
144  ●    SELECT c.full_name, o.order_id
145      FROM customers c
146      LEFT JOIN orders o ON c.customer_id = o.customer_id
147      UNION
148      SELECT c.full_name, o.order_id
149      FROM customers c
150      RIGHT JOIN orders o ON c.customer_id = o.customer_id;

```



Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	full_name	order_id
▶	Aman Verma	1
	Rahul Sharma	2
	Sneha Patil	3
	Pooja Singh	4
	Rohit Jain	5
	Neha Mehta	6



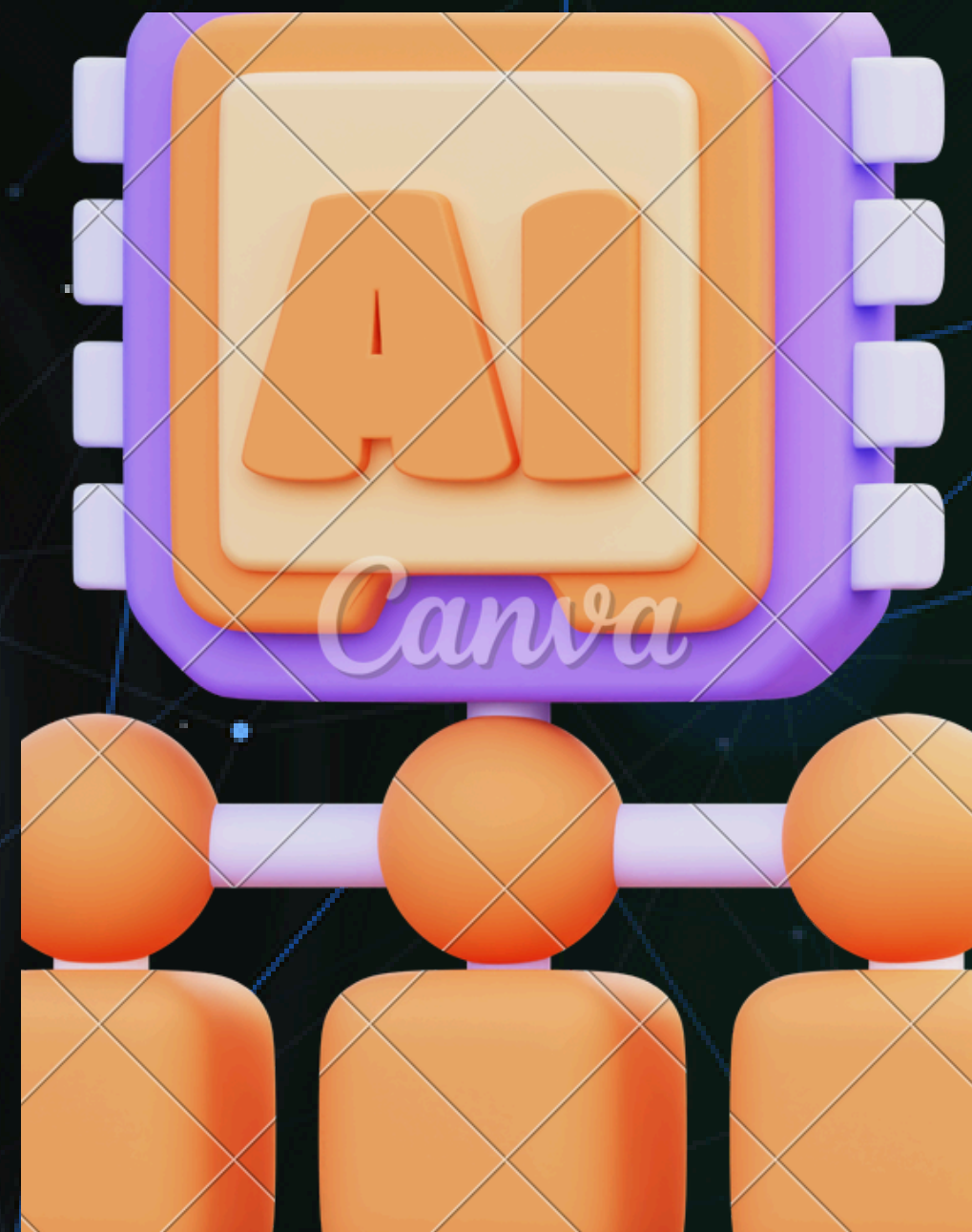
```
52 -- WHERE CLAUSE
53 -- display mobiles whose price is greater than 20000.
54 • SELECT *
55 FROM mobiles
56 WHERE price > 20000;
```

[illegible]

```
85 -- COUNT
86 -- count total number of customers.
87 • SELECT COUNT(*) AS total_customers
88 FROM customers;
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content:

total_customers
10



```
190
191 -- GROUP BY
192 -- count number of customers in each city.
193 • SELECT city, COUNT(*) AS customer_count
194 FROM customers
195 GROUP BY city;
196
197
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

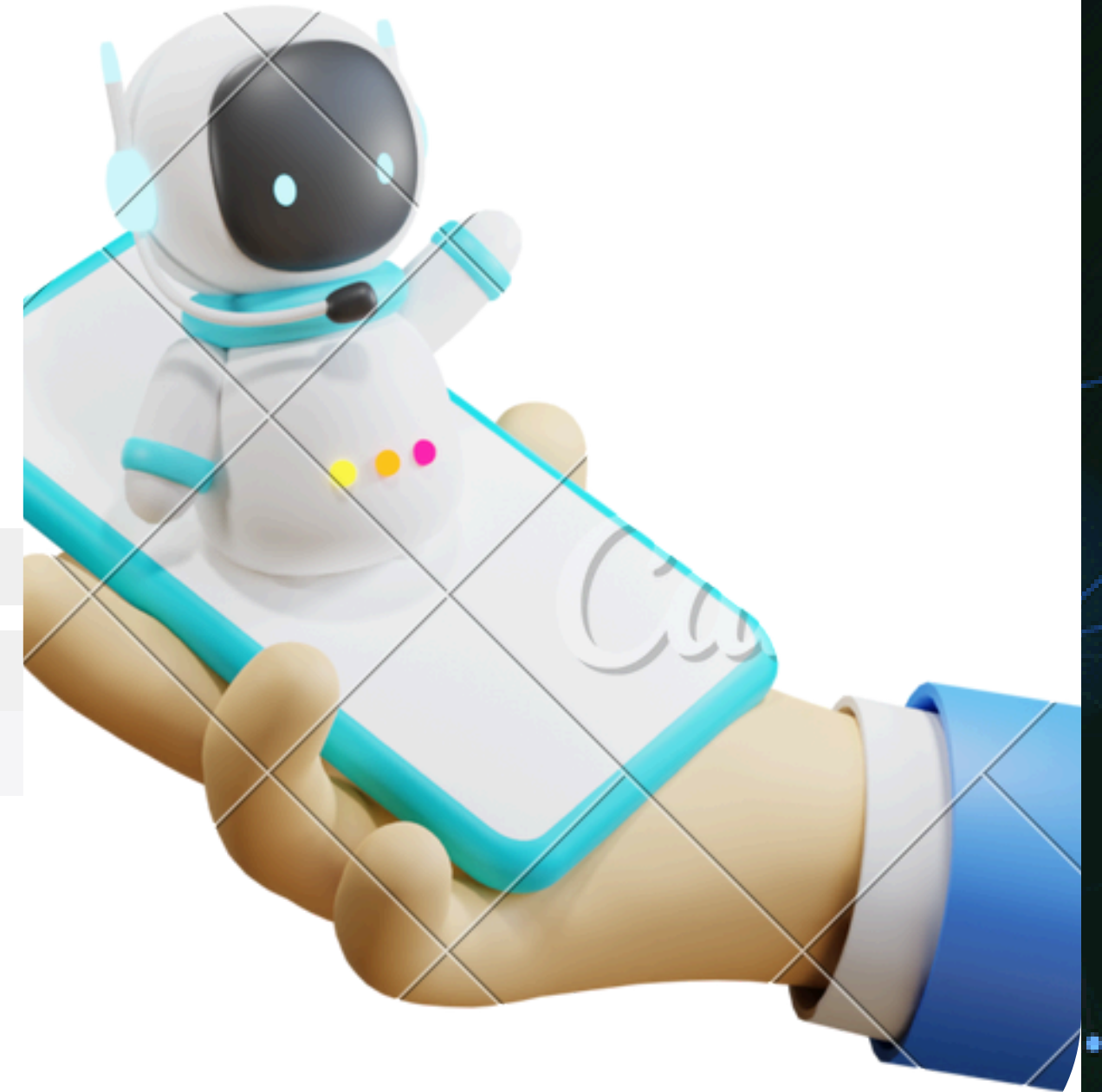
city	customer_count
Delhi	2
Mumbai	1
Pune	1
Nagpur	1
Bhopal	1
Indore	1

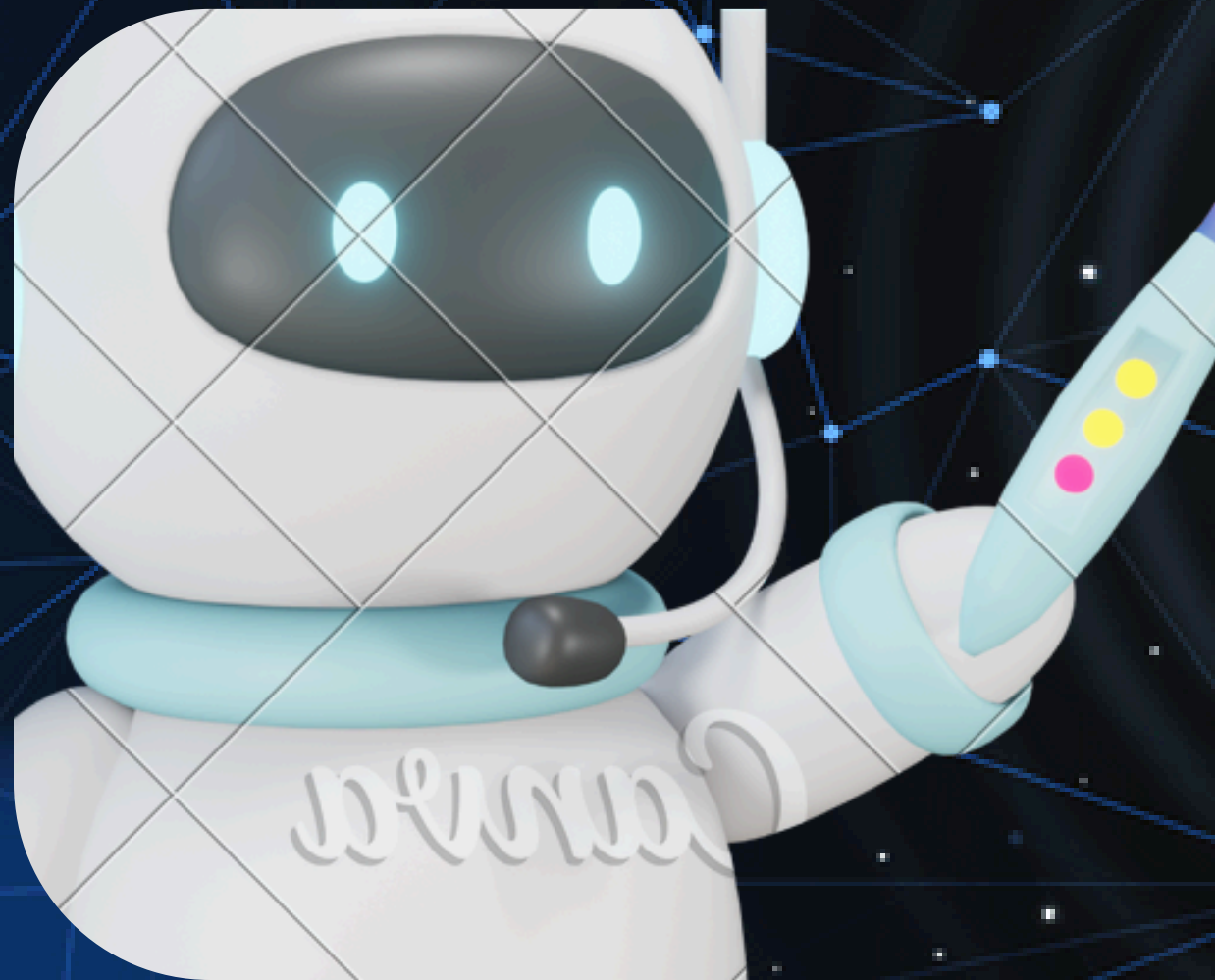


```
214
215  -- HAVING CLAUSE
216  -- display cities having more than one customer.
217 • SELECT city, count(*) AS customer_count
218     FROM customers
219     GROUP BY city
220     HAVING COUNT(*) > 1
221     LIMIT 5;
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	city	customer_count
+	Delhi	2





```
197
198      -- AVG
199      -- find the average price of mobiles.
200 •    SELECT AVG(price) AS average_price
201      FROM mobiles;
202
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



average_price
27650.0000



```
202
203
204      -- SUM
205      -- calculate total sales amount.
206 •  SELECT SUM(total_amount) AS total_sales
207      FROM orders;
208
209
210      -- MAX
211      -- find the maximum mobile price.
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

	total_sales
▶	276500



```
209
210      -- MAX
211      -- find the maximum mobile price.
212  •   SELECT MAX(price) AS max_price
213      FROM mobiles;
214
```



Result Grid



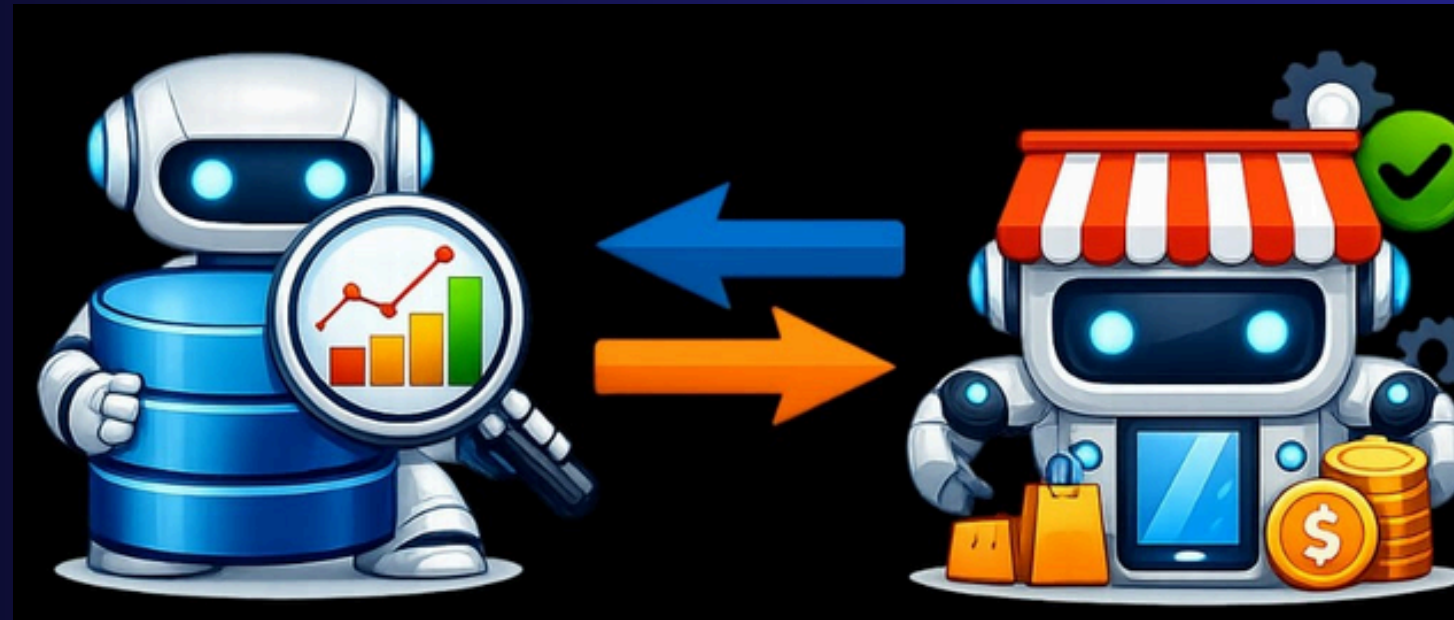
Filter Rows:

Export:



Wrap Cell Content:

	max_price
▶	60000



INSIGHTS

From this Mobile Shop database, useful information such as customer purchases, total sales, popular mobiles, and employee involvement can be easily obtained.

Using JOIN queries helps in analyzing data from multiple tables efficiently.

CONCLUSION:

The Mobile Shop Management System is a simple SQL-based project that organizes data using primary and foreign keys. It makes mobile shop data handling easier, faster, and more accurate

THANK
YOU